FACULITY OF COMPUTING TECHNOLOGY

Deoartment of computer science

Title of project: – Minimum Spanning Tree for Rural Internet Connectivity Using Kruskal's Algorithm

Group members

1. Tsgabu girmay------Aku1602142
2. Genet yared---------Aku1602333
3. Legesse baraki------Aku1601920

SUBMITTED TO: - EKUBAZGI G.

March 30, 2026

Contents

Minimum Spanning Tree for Rural Internet Connectivity Using Kruskal's Algorithm 0

Title Page ..... **Error! Bookmark not defined.**

1. Introduction – Real-World Problem Context 2

2. Problem Understanding 2

3. Graph Representation 2

4. Algorithm Design 3

Kruskal's Algorithm Steps 3

5. Pseudocode 3

6. Graph Implementation 3

Data Structures Used 3

Distance Calculation 3

7. Dataset Description 4

Dataset Characteristics 4

8. Step-by-Step Execution (Small Graph Example) 4

9. Implementation Details 5

10. Time Complexity Analysis 5

11. Space Complexity Analysis 6

12. Correctness Proof 6

13. Experimentation and Results 6

14. Real-World Significance 7

15. Challenges Faced 7

16. Conclusion 8

17. References 8

1. Introduction – Real-World Problem Context

Providing reliable internet connectivity to rural and remote villages is a major challenge for governments and telecommunications providers. One of the most expensive components of network deployment is laying fiber optic cables between villages. Since budgets are limited, it is important to design the network in a way that minimizes the total cost while ensuring that every village is connected either directly or indirectly.

This problem can be modeled using graph theory. In this model:

- Villages are represented as **vertices**
- Cable connections are represented as **edges**
- Distance between villages represents the **cost (weight)**

The objective is to connect all villages using the minimum total cable length without creating cycles. This type of problem is known as the **Minimum Spanning Tree (MST)** problem.

To solve this problem efficiently, this project uses:
Kruskal's Algorithm with Union-Find (Disjoint Set).

The system generates a network of 100 villages, calculates distances using the Euclidean distance formula, and constructs the optimal network using arrays and manually implemented sorting.

2. Problem Understanding

The government wants to:

- Connect all villages
- Use the minimum total cable length
- Ensure the network has no cycles
- Maintain connectivity

This is a classic **Minimum Spanning Tree problem**.

3. Graph Representation

The network is represented as:

Graph G = (V, E)

Where:

- **V = Set of villages**
- **E = Set of possible cable connections**
- Each connection has a **weight = distance between villages**

4. Algorithm Design

Selected Algorithm: Kruskal's Algorithm Why Kruskal's Algorithm Was Chosen

Kruskal's algorithm was selected because:

1. It is simple to implement
2. It works efficiently with large graphs
3. It naturally uses arrays
4. It guarantees the minimum cost solution
5. It allows cycle detection using Union-Find

Kruskal's Algorithm Steps

1. Generate all possible edges
2. Sort edges in ascending order of weight
3. Initialize Union-Find structure
4. Select the smallest edge
5. Check if it creates a cycle
6. If no cycle, add edge to MST
7. Repeat until V − 1 edges are selected

5. Pseudocode

```
KruskalMST(Graph G):
    Sort all edges by weight
    Initialize Union-Find
    For each edge in sorted list:
        If Find(source) ≠ Find(destination):
            Add edge to MST
            Union(source, destination)
    Stop when MST contains V - 1 edges
```

6. Graph Implementation

Data Structures Used

Arrays were explicitly used as required.

- Village Array: Village villages[MAX_VERTICES]
- Edge Array: Edge edges[MAX_EDGES]
- MST Array: Edge mstEdges[MAX_VERTICES - 1]
- Union-Find Arrays:
  - int parent[MAX_VERTICES]
  - int rank[MAX_VERTICES]

Distance Calculation

Euclidean distance formula:

Distance = $\sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$

This calculates the cable length between two villages.

7. Dataset Description

The dataset was generated using random coordinates.

Dataset Characteristics

- Number of villages: **100**
- Coordinates range: **0 to 100 km**
- Graph type: **Fully connected graph**

Total possible edges:

E = V(V − 1) / 2
E = 100 × 99 / 2
E = 4950 edges

The graph is guaranteed to be connected.

8. Step-by-Step Execution (Small Graph Example)

Example villages:
Alpha, Beta, Gamma, Delta, Epsilon, Zeta

Step 1 — Generate All Edges

Example:

- Alpha — Beta
- Beta — Gamma
- Gamma — Delta

Step 2 — Sort Edges by Distance

Smallest edges first:

- Alpha — Beta → 5 km
- Beta — Gamma → 6 km
- Gamma — Delta → 7 km

Step 3 — Add Edges without Cycles

Add:

- Alpha — Beta
- Beta — Gamma
- Gamma — Delta

Skip:

- Alpha — Gamma (because it creates a cycle)

Step 4 — Stop When:

Number of edges = V − 1
For 6 villages → Edges = 5

9. Implementation Details

Key Functions

1. **Add Village**
   addVillage(id, x, y, name)
   Adds a new village to the network.
2. **Generate Edges**
   generateEdges()
   Creates all possible connections.
3. **Sort Edges**
   Manual Bubble Sort implemented:
   sortEdges()
   No built-in sorting functions used.
4. **Kruskal Algorithm**
   kruskalMST()
   Builds the Minimum Spanning Tree.
5. **Verify MST**
   verifyMST()
   Checks:
   - Number of edges = V − 1
   - Graph is connected
   - No cycles
6. **Save Results**
   saveResultsToCSV()
   Outputs: mst_results.csv

10. Time Complexity Analysis

Let:

- V = Number of vertices
- E = Number of edges

Sorting Edges

O(E log E)

Union-Find Operations

O(E α(V))
Where α(V) is the inverse Ackermann function (almost constant).

Total Time Complexity

O(E log E)

Example:

- V = 100
- E = 4950

Time Complexity ≈ **O(4950 log 4950)**

11. Space Complexity Analysis

The program stores:

- Village array → O(V)
- Edge array → O(E)
- Union-Find arrays → O(V)

Total Space Complexity O(V + E)

12. Correctness Proof

The algorithm produces a correct Minimum Spanning Tree because:

1. **Acyclic** → Union-Find prevents cycles
2. **Connected** → All villages are connected
3. **Minimum Cost** → Edges selected in increasing order
4. **Edge Count** → Number of edges = V − 1

These are required properties of MST.

13. Experimentation and Results

Small Graph

- Villages: 6
- Edges in MST: 5
- Result: Valid MST

Large Graph

- Villages: 100
- Possible edges: 4950
- MST edges: 99
- Result: Valid MST

Output Example

- Number of Villages: 100
- MST Connections: 99
- Total Cable Length: 742.35 km
- MST Verification: Valid (Connected and Acyclic)

14. Real-World Significance

This project demonstrates how algorithms can solve real infrastructure problems.

Applications include:

- Rural internet connectivity
- Electrical power grid design
- Road network planning
- Water distribution systems
- Telecommunications networks

Using Minimum Spanning Tree algorithms can:

- Reduce infrastructure cost
- Improve efficiency
- Optimize network planning
- Support government development programs

15. Challenges Faced

Some challenges encountered include:

- Handling large numbers of edges
- Ensuring the graph is connected
- Implementing sorting manually
- Designing Union-Find correctly
- Managing memory using arrays

16. Conclusion

This project successfully implemented Kruskal's algorithm to design a minimum-cost rural internet connectivity network. The system generated a graph of 100 villages, calculated distances, sorted edges, and constructed the optimal network using arrays and Union-Find.

The results confirmed that:

- All villages were connected
- No cycles were formed
- The total cable length was minimized

This demonstrates the practical importance of graph algorithms in solving real-world engineering and infrastructure problems.

17. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, MIT Press
- Goodrich, M. T., Tamassia, R. *Data Structures and Algorithms in C++*, Wiley
- GeeksforGeeks – Minimum Spanning Tree (Kruskal's Algorithm)
- Kaggle – Village Geographic Data
- ISO C++ Documentation – https://cplusplus.com